SLIDE 1

@java_interview_prep



Java 11

Complete Interview Guide



Most Asked Java Version in Enterprise Applications



Important Features & Interview Questions



Handwritten Notes



Save Before Your Interview

SLIDE 2

@java_interview_prep

Why Java 11?

Why is Java 11 important?

Java 11 is an LTS (Long-Term Support) release.

Java 11

- Released in 2018
- LTS version
- Production-friendly
- Improved APIs
- HTTP Client API
- String API improvements
- var in Lambda parameters
- Modern JVM improvements





Interview Tip

Java 8 → Java 11 is one of the most common upgrade discussions in enterprise projects.

SLIDE 3

@java_interview_prep

Java 11 Features

Must-Know Features

1. New HTTP Client API — HTTP
2. String. isBlank()
3. String. lines()
4. String. strip()
5. String. repeat() — "Java".repeat(3)
6. Files. readString()
7. Files. writeString()
8. Optional. isEmpty() — Optional
9. var in Lambda parameters — (var a, var b) → a + b
10. Single-file source-code execution — > java Hello.java

Interview Focus

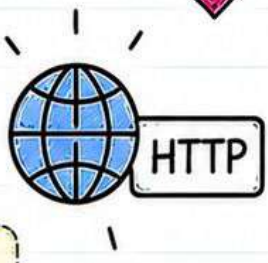
Don't just remember the methods — know why they were introduced and how they differ from older approaches.

Save Before Your Interview

SLIDE 4

@java_interview_prep

New HTTP Client API



Java 11 introduced the modern HTTP Client API which is simple, powerful and supports HTTP/2.

Example (GET Request)

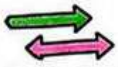
```
HttpClient client = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create("https://api.example.com/users"))
        .GET()
        .header("Accept", "application/json")
        .build();
HttpResponse<String> response =
    client.send(request, HttpResponse.BodyHandlers.ofString());
// Response Body
System.out.println(response.body());
```

Clean, Readable and Powerful!

Supports:

- HTTP/1.1 and HTTP/2
- Synchronous & Asynchronous calls
- WebSocket support
- Request timeout, redirects, cookies, proxy
- Built-in support for SSL/TLS











Interview Question

Q Why should we prefer Java 11 HttpClient over HttpURLConnection?

A Modern API, HTTP/2 support, async calls, better performance, and cleaner code.



Save Before Your Interview



SLIDE 5

@java_interview_prep

New String Methods

Java 11 added several powerful methods to the String class.

1 isBlank() → Checks if a string is empty or contains only whitespace.

```
"   ".isBlank();   // true
"Hello".isBlank(); // false
```

Tip

isBlank() is better than isEmpty() when you want to check for blank spaces as well.

2 lines() → Returns a Stream of lines split from the string.

```
"Hello\nJava\n11".lines()
    .forEach(System.out::println);
```

Output

- Hello
- Java
- 11



3 strip() → Removes leading and trailing whitespace (Unicode aware).

```
"  Hello ".strip(); // "Hello"
```

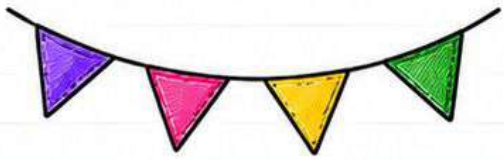
→ VS

strip() vs trim()

- trim() → Removes whitespace only up to \u0020
- strip() → Removes all Unicode whitespace characters

4 repeat(n) → Returns a string that is repeated n times.

```
"Java".repeat(3);  // "JavaJavaJava"
```



Quick Summary

- ✓ isBlank() → Blank or whitespace
- ✓ lines() → Stream of lines
- ✓ strip() → Unicode-aware trim
- ✓ repeat(n) → Repeat the string

Save This Slide Before Your Interview

SLIDE 6

@java_interview_prep

Files API Improvements



Java 11 introduced new methods in Files class to simplify reading and writing text files.

Before Java 11

Reading a file

```
byte[] bytes = Files.readAllBytes(path);
String data = new String(bytes,
              StandardCharsets.UTF_8);
```

Writing to a file

```
byte[] bytes = data.getBytes(
               StandardCharsets.UTF_8);
Files.write(path, bytes,
 StandardOpenOption.CREATE,
 StandardOpenOption.TRUNCATE_EXISTING);
```

Java 11 Way

Reading a file

```
String data = Files.readString(path);
```

Reads the entire file content as a String (UTF-8 by default)

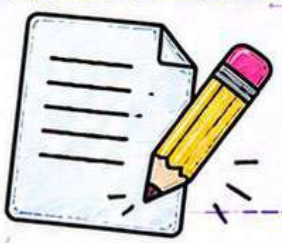
Writing to a file

```
Files.writeString(path, data);
```

Writes string to file using UTF-8 (Create or overwrite by default)

Benefits

- Less boilerplate code
- More readable and clean
- Perfect for text files that fit in memory
- Uses UTF-8 by default (good for international text)



Note

For very large files, use Streams instead of readString().

Interview Tip

These methods are convenient for text files that can be reasonably handled as a whole in memory.

Methods

- Files.readString(Path)
- Files.writeString(Path, String)
- Files.exists(Path) :
- Files.notExists(Path)

Save Before Your Interview

SLIDE 7

@java_interview_prep

Optional & Lambda Improvements



Java 11 added useful improvements to Optional and enhanced Lambda parameters.

1 Optional

Java 11 added isEmpty() to Optional.

```
Optional<String> name = Optional.empty();
if (name.isEmpty()) {
    System.out.println("No value present");
}
```

Before Java 11

```
if (!name.isPresent()) {
    System.out.println
      ("No value present");
}
```

isEmpty() is just the opposite of isPresent(). It improves readability.

2 var in Lambda Parameters

Java 11 allows usage of var in lambda parameters.

```
(var a, var b) → a + b
```

Why useful?

- Helps when annotations are applied to parameters.
- Avoids repeating long types.

Example with Annotations

```
Function<String, String> fn =
  (@NotNull var name) →
    name.trim();
```

Interview Tip

- var in lambda is only for parameters.
- It is not allowed for method parameters or fields.
- Introduced in Java 10 for local variables, expanded to lambda parameters in Java 11.

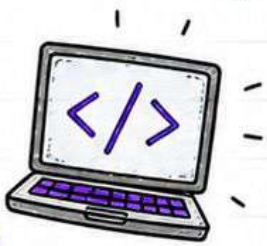


Save This Slide Before Your Interview

SLIDE 8

@java_interview_prep

Single-File Source Execution



Java 11 allows you to run a single Java source file directly without creating a .class file explicitly.

Before Java 11

We had to compile and then run.

Steps:

1. Write Hello.java
2. Compile using javac
3. Run the .class file using java

Example:

```
$ javac Hello.java
$ java Hello
```

Java 11 Way

Run the source file directly.

Example:

```
$ java Hello.java
```

How it works?

The JVM compiles the source code in memory and runs it.
No .class file is created on disk.

Why is it useful?

- ✓ Quick experiments
- ✓ Small programs / scripts
- ✓ Learning new concepts
- ✓ Prototyping ideas

Important Notes

- Works only for a single source file with a public class.
- If the file has multiple classes, only one public class is allowed.
- It is still compiled by the JVM, not interpreted.
- Not meant to replace the normal compilation process used in large projects.

Interview Tip

Interviewers may ask:
"Does Java 11 become an interpreted language?"
Answer: No. It still compiles the code,
but compilation happens in-memory and is not saved as .class.

Save Before Your Interview

SLIDE 9

@java_interview_prep

Most Asked Interview Traps



1 Q. Is Java 11 an LTS version?

A. Yes. Java 11 is an LTS (Long-Term Support) release.



2 Q. What is the biggest API addition?

A. Java 11 HTTP Client API.



3 Q. trim() vs strip()?

A. strip() is Unicode-aware and removes whitespace from the beginning and end more accurately than trim().

```
"   Hello   ".trim()  → "Hello"
"   Hello   ".strip() → "Hello"
(Works better with Unicode spaces)
```



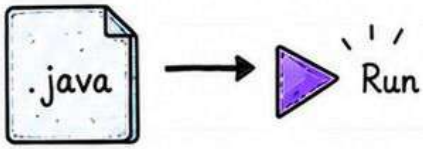
4 Q. isBlank() vs isEmpty()?

A.
- isEmpty() → length == 0
- isBlank() → empty OR contains only whitespace

```
" ".isEmpty()    → true
" ".isBlank()    → true
"  ".isEmpty()   → false
"  ".isBlank()   → true
```

5 Q. Can Java 11 run a .java file directly?

A. Yes. You can run a single Java source file directly using: java File.java



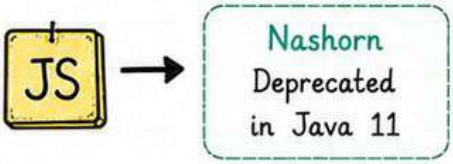
6 Q. Does Java 11 support var in method parameters?

A. No. var is NOT allowed in method parameters. It is allowed only in lambda parameters (since Java 11).

```
✗ void method(var x) { }
✓ (var x, var y) → x + y
```

7 Q. What happened to Nashorn?

A. Nashorn JavaScript engine was deprecated in Java 11 and removed in later versions (from Java 15).





Interview Tip

Don't just remember the features, understand the WHY behind them!



Save Before Your Interview



SLIDE 10

@java_interview_prep

Quick Revision

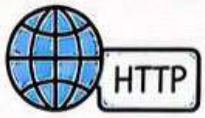
Java 11

✓ Revise
✓ Remember
✓ Rock Your Interview

Let's quickly revise the most important Java 11 features!

1 New HTTP Client API

- Modern, async, non-blocking
- Better performance & easy to use



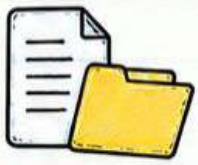
2 New String Methods

- isBlank(), lines(), strip(), repeat(n)
- More powerful and readable string handling



3 Files API Improvements

- readString(), writeString()
- Read/Write files easily with UTF-8 by default



4 Optional Enhancements

- isEmpty() added
- Better checks for empty values

Optional<String>
isEmpty()

5 Lambda Improvements

- var in lambda parameters
- Type inference makes code cleaner

(var a, var b) → a + b

6 Single-File Source Execution

- Run a .java file directly
- Great for small programs and quick testing

```
$ java Hello.java
```

7 Removed & Deprecated Features

- Nashorn JavaScript engine removed
- Some old APIs deprecated

